

Отчёт по лабораторной работе № 6 ▾

Название работы: Интерактивный rebase, cherry-pick, теги, reflog

Студент: Блинов Михаил Андреевич

Группа: АТ-13

Дата выполнения: 17.05.2026 📅

1. Выполнение шагов лабораторной работы

Отметьте галочками выполненные шаги. В столбце «Команды / вывод» напишите выполненные команды и полученный вывод (поле автоматически растягивается по содержимому).

Шаг	Описание	Вып.	Команды / вывод
1	Настроить редактор Git на nano (git config --global core.editor 'nano')	☒	git config --global core.editor "notepad" -> В качестве глобального текстового редактора для Git успешно установлен Блокнот
2	Создать репозиторий и сделать 5 коммитов (Step 1..5)	☒	1..5 ForEach-Object { "Commit \$_" > "f\$_.txt"; git add .; git commit -m "Step \$_" } -> Инициализирован репозиторий и в цикле последовательно создано 5 базовых коммитов
3	Создать ветку feature-rebase, добавить три коммита (Feature A, Feature B, Fix typo)	☒	git checkout -b feature-rebase -> Создана отдельная ветка feature-rebase, в которой сделано 3 коммита с фичами и исправлением опечатки
4	Запустить git rebase -i HEAD~3 и изменить: squash для Feature B, reword для Fix typo	☒	git rebase -i HEAD~3 -> Интерактивный rebase запущен, в списке коммитов для Feature B указан squash, а для Fix typo — reword
5	Сохранить изменения в редакторе (nano: Ctrl+O,	☒	Ctrl+S (в Блокноте) -> Список коммитов сохранен, запущен процесс

Шаг	Описание	Вып.	Команды / вывод
	Enter, Ctrl+X)		объединения и изменения истории
6	При squash объединить сообщения коммитов, при reword написать 'Fix critical typo in f3.txt'	✓	Fix critical typo in f3.txt -> Сообщения коммитов успешно объединены, для исправленного коммита задано новое описание
7	Проверить историю (git log --oneline) — коммитов стало меньше	✓	git log --oneline -> Successfully rebased and updated. История ветки feature-rebase переписана, количество коммитов уменьшилось за счет squash
8	Переключиться в main и скопировать исправленный коммит через cherry-pick	✓	git checkout main && git cherry-pick d432280 -> Осуществлен переход в главную ветку, коммит с исправлением успешно перенесен по его хэшу
9	Создать лёгкий тег v1.0.0 и аннотированный тег v1.1.0	✓	git tag v1.0.0 && git tag -a v1.1.0 -m "Аннотированный тег" -> Созданы два типа меток на последнем коммите: легковесный тег v1.0.0 и аннотированный тег v1.1.0
10	Переключиться по тегу v1.0.0 (detached HEAD), создать от него ветку hotfix-from-tag	✓	git checkout v1.0.0 && git checkout -b hotfix-from-tag -> Осуществлен переход на тег в режим detached HEAD, и от этой точки успешно создана новая ветка hotfix-from-tag
11	Выполнить git reset --hard HEAD~3, затем восстановить потерянные коммиты через reflog	✓	git reset --hard HEAD~3 && git reflog && git reset --hard HEAD@{1} -> История коммитов принудительно сброшена на 3 шага назад, а затем полностью восстановлена в исходное состояние через журнал reflog

2. Самостоятельное задание

Условие:

Создайте ветку messy, сделайте 4 коммита. С помощью git rebase -i HEAD~4 объедините первые два коммита и переименуйте третий. Перенесите один коммит из messy в main через cherry-pick.

Создайте аннотированный тег exercise-v1.

Выполнение (ход действий и результат):

```
git checkout -b messy
```

```
1..4 | ForEach-Object { "Messy $_" >> f.txt; git add .; git commit -m "Messy $_" }
```

(Создана ветка messy, в ней сделано 4 коммита).

```
git rebase -i HEAD~4
```

(В Блокноте для первого коммита оставлен pick, для второго указан squash, для третьего reword, история успешно переписана).

```
git checkout main
```

```
git cherry-pick messy
```

(Объединенный коммит успешно перенесен из ветки messy в main).

```
git tag -a exercise-v1 -m "Exercise tag"
```

(Создан аннотированный тег exercise-v1 на вершине main).

3. Ответы на контрольные вопросы

1. Вопрос 1:

Rebase изменяет хэши коммитов, фактически создавая их копии. Если ветка уже была отправлена в публичный репозиторий, rebase перезапишет историю на сервере. Это ломает синхронизацию для всех остальных разработчиков, что приведет к запутанным конфликтам при попытке стянуть изменения

2. Вопрос 2:

git cherry-pick копирует в текущую ветку изменения только одного конкретного коммита по его индивидуальному хэшу. git merge сливает всю последовательность коммитов выбранной ветки целиком — от точки их разделения до самого последнего коммита

3. Вопрос 3:

Нужно выполнить команду git reflog, которая хранит историю всех перемещений указателя HEAD. В списке необходимо найти хэш-код коммита, существовавшего до выполнения сброса, и вернуть историю командой git reset --hard <хэш>

4. Дополнительные замечания (при необходимости)

Инструкция: Выберите номер лабораторной работы. Заполняйте поля — они автоматически растягиваются по высоте под текст. Для печати нажмите кнопку «Печать» — все рамки полей исчезнут, и отчёт будет выглядеть как обычный документ.